

Conv-inheritance: A hardware-efficient method to compress convolutional neural networks for edge applications

Yang Yang^a, Chao Wang^{a,*}, Lei Gong^a, Min Wu^b, Xuehai Zhou^a

^a School of Computer Science and Technology, University of Science and Technology of China, Hefei, China

^b Institute for Infocomm Research, A*STAR, Singapore

ARTICLE INFO

Article history:

Received 30 November 2020

Revised 4 February 2021

Accepted 25 February 2021

Available online 4 November 2021

Keywords:

Convolutional neural network

Compression

Hardware-efficient

ABSTRACT

Convolutional Neural Networks (CNNs) have won tremendous success in various applications, such as image recognition and natural language processing. Due to the computing-intensive and memory-intensive features of CNN models, it is challenging to deploy them on devices with limited resources and tight power budgets. To address this limitation, pruning, quantization, weights sharing, and other methods have been proposed as efficient solutions to compress CNN models. However, these works usually do not optimize the hardware design when compressing the CNNs. In this paper, we introduce Conv-Inheritance, a hardware-efficient compression method for CNNs by reducing the number of convolution operations to simultaneously reduce the inference time and hardware computing resources on chip of CNN edge devices. We also develop an extended version, called adaptive Conv-Inheritance, to further improve the inference performance by considering the similarity between pixels. We applied our Conv-Inheritance method to compressing various CNN models and performed extensive experiments on different image datasets. Experimental results demonstrate that Conv-Inheritance is both hardware-efficient and time-efficient on edge devices. Besides, Conv-Inheritance is a plug-and-play compression method for CNNs, and we can run Conv-Inheritance on top of other compression methods without any conflicts.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

In recent years, Convolutional Neural Networks (CNNs) are widely adopted in various applications, such as object detection [1], mobile robot vision [2], image search engine, etc. Along with the evolution of CNN models, a general trend is to scale up both network size (i.e., the number of layers and neurons) [3] and input resolution [4]. From AlexNet [5], VGGNet [6] and GoogleNet [7] to ResNets [8], the models winning the ImageNet Classification Challenge have evolved from 8 layers to more than 100 layers. However, larger CNNs, which have more powerful learning ability, also raise growing computation demand and storage resource requirement for hardware [9]. For instance, a 152-layer ResNet has more than 20 Giga float-point-operations (FLOPs) to make a prediction for an image with resolution 224×224. Even with CNN customized accelerators, it is still very challenging to deploy those large and complex CNN models on edge devices [33].

To deal with these issues, model compression techniques have been proposed to compress deep neural networks for edge deployment. For instance, parameter quantization methods compress the original network by using fewer bits to represent the weights [10]. Transforming the operations of convolution [11] changes the traditional vanilla convolution into an Octave convolution operation. Another commonly used method for model compression is weight pruning [12], which aims to remove unnecessary parameters in a trained deep neural network. Although the weight pruning is able to reduce model storage size, it increases the complexity of storage and data interconnection on hardware platforms [13]. Other methods like weights sharing [14] have also shown the capability of reducing the model size, but they only address the storage complexity issue of deep models and can not reduce the computing resource demand [15]. All these methods have obtained notable achievements in compressing CNNs to deploy them on resource-limited hardware systems.

Nevertheless, all these model compression methods mentioned above mostly focus on the compression of CNN models without considering hardware and software co-design. These compression

* Corresponding author.

E-mail address: cswang@ustc.edu.cn (C. Wang).

methods do not consider the detailed design on chip for compressing the CNN models and require lots of effort to redesign the hardware for deploying them. For instance, when the sparsity of fully connected layer weights pruning is not large enough, the storage and communication overhead on edge devices can be more than the CNNs deployment without pruning. Moreover, transforming the convolution operations takes a very long time and much effort to redesign the computing unit on chip of convolution operations and storage and communication paths of edge devices.

In this paper, we propose Conv-Inheritance, a hardware-efficient CNN compression method, which tackles the redundancy of CNNs and optimizes detailed design on chip. Our approach takes convolution operation redundancy of CNN models as the compression target. In image processing, pixel inheritance is a method of changing pixels in pictures and makes the changes hard to distinguish by making a pixel inherit the pixel value of the front pixel. Inspired by pixel inheritance, Conv-Inheritance reduces the convolution operations by using the front pixel value as the convolution result so that a convolution computing unit on chip can be replaced with data reuse easily. Thus, it takes very little effort to deploy Conv-Inheritance on edge devices. An adaptive Conv-Inheritance, as an extended version of Conv-Inheritance, is further developed by considering the similarity between different image pixels. In particular, two neighboring pixels are considered to be similar if the pixel difference between them is less than the threshold. Then, we utilize the Conv-Inheritance to gain the output value. Moreover, Conv-Inheritance is a plug-and-play compression method for CNNs. It can be used directly in the forward propagation of the convolution operations without any adjustments in the network architecture or other processes' functions. Hence, there are no conflicts between Conv-Inheritance and other compression methods [16]. Given that many deep learning frameworks transform the computation of convolution operation to the heavily-optimized matrix multiplications for high computational performance, we propose the matrix rebuild algorithm of Conv-Inheritance for deep learning frameworks.

For efficient hardware deployment, Conv-Inheritance can be a customized compression method for specialized edge devices and applications. When deploying Conv-Inheritance on the edge devices, there are four key factors to consider. They are the pixel similarity of inputs, computing resources on given hardware platforms, accuracy demand for applications, and other compression methods utilized together. The first factor is positively associated with the inheritance ratios (the percentage of Conv-Inheritance pixels). And the last three factors are negatively associated with the inheritance ratios. With the comprehensive consideration of the above four factors, edge devices achieve high performance via customized Conv-Inheritance.

To summarize, the contributions of our work are as follows:

- We propose Conv-Inheritance as a plug-and-play and hardware-efficient CNN compression method for reducing convolution operation redundancy. For efficient hardware deployment, Conv-Inheritance can be a customized compression method for specialized edge devices and applications.
- We propose the basic algorithm of Conv-Inheritance and the matrix rebuild algorithm of Conv-Inheritance to achieve high computational performance. We also develop an extended version, called adaptive Conv-Inheritance, to further improve the inference performance by considering the similarity between pixels.
- Experimental results show that Conv-Inheritance can reduce resource requirements of hardware and gain speedup of convolution operations, demonstrating that Conv-Inheritance can be both hardware-efficient and time-efficient on edge devices.

2. Related Work

Compressing CNNs has been a popular research topic in recent years. The current works include model quantization [12], network pruning, and weight sharing [17]. These works have made some improvements based on existing CNN models.

Others are to directly hand-craft more efficient architectures for edge devices. In [18], SqueezeNet reduces the number of parameters and computation by using lower-cost 1×1 convolutions and reducing filter sizes. MobileNet [19] extensively employs depth-wise separable convolution to minimize computation density. ShuffleNet [20] utilizes low-cost group convolution and channel shuffle. CondenseNet [21] learns to connect group convolutions across layers. Recently, MobileNetV2 [22] achieves state-of-the-art results among mobile-size models by using resource-efficient inverted residuals and linear bottlenecks. However, these hand-crafted models usually require significant human efforts.

In the last several years, there is an increasing interest in automating the model design process using neural architecture search. These works are mainly based on reinforcement learning [23], evolutionary search [24], differentiable search [25], and other learning algorithms [26]. For example, reinforcement learning uses policy networks to guide the next-step architecture component [23]. The evolution processes guide the mutation and recombination process of candidate architectures [24]. These works mostly cost plenty of time and hardware resources for training repeatedly.

Knowledge Distillation (KD), also known as the teacher-student strategy, is also widely applied to CNN model compression. It was firstly introduced by [27], which refers to training a shallow network (i.e., Student) by mimicking the output of a more extensive and deeper network (i.e., Teacher). Hinton et al. further generalized it by introducing a temperature variable to soften the logits from the cumbersome model as "soft target" [28]. For excellent knowledge Distillation, there has to be a strong teacher network, and the design of the student network still takes lots of human efforts [34].

All the works mentioned above have achieved notable success in slimming the model architecture and reducing the weights of CNNs. However, they do not consider the hardware efficiency and optimizing the deployment on edge devices. We are thus motivated to optimize the detailed design on chip for efficient deployment on edge devices.

3. Methodology

3.1. Conv-Inheritance

Biologically, the human eye is not a very sensitive visual organ. It cannot distinguish the change of one-pixel point in the whole picture since neighboring pixels are usually similar. Pixel inheritance is thus proposed to change pixels and make the changes hard to distinguish. In pixel inheritance, a pixel inherits the pixel value of the front pixel. For instance, if we want to change half of the pixels by pixel inheritance, we make the behind pixel value equal to the front in every two neighboring pixels. Then the two pixels share the same pixel value. When processing the actual images, we often make the whole column or the row inherit the front. Fig. 1 shows a series of pictures processed by pixel inheritance.

Inspired by pixel inheritance, we propose Conv-Inheritance to gain the value of pixels in output images to reduce the convolution operation redundancy. There are three kinds of basic Conv-Inheritance methods: column inheritance, row inheritance, and crossing inheritance. In the column inheritance, we choose one column from the output to inherit the front column's pixel values. Similarly, in the row inheritance, we choose one row to inherit



Fig. 1. Pixel Inheritance.

the front row's pixel values. When it comes to crossing inheritance, we choose both one row and one column to inherit the front's pixel values.

Fig. 2 shows examples of Conv-Inheritance in three different kinds of methods. The percentage under the picture means the percentage of the pixels in the picture inheriting the front. The pixels in the shadows are the rows and the columns in which we utilize Conv-Inheritance to obtain the pixel values instead of normal convolution operations.

Besides, Conv-Inheritance is a plug-and-play compression method for CNNs. It can be used directly in the forward propagation of the convolution operations without any adjustments in the network architecture or other processes' functions.

3.2. Customized Conv-Inheritance

For efficient hardware deployment, Conv-Inheritance can be a customized compression method for specialized edge devices and applications. When deploying Conv-Inheritance on the edges

devices, there are four key factors to consider. They are the pixel similarity of inputs, computing resources on given hardware platforms, accuracy demand for applications, and other compression methods utilized together.

The pixel similarity means how many pixels in pictures share the same pixel value or almost the same. As we talked about before, Conv-Inheritance is based on the similarity between neighboring pixels and the insensitivity of human eyes. If the input picture has many of the similarity of pixels, Conv-Inheritance works well. Computing resources are always limited on edge devices. However, there is no good to save resources as much as possible. The best running state is making full use of all the resources. Thus, when determining the Conv-Inheritance ratio, the best ratio is making the best use of the resources on edge devices.

Accuracy demand varies for different applications. The higher accuracy demand leads to a lower inheritance ratio. As there are no conflicts between Conv-Inheritance and most other compression methods, we can utilize Conv-Inheritance together with other compression methods. Because the CNN model's redundancy is

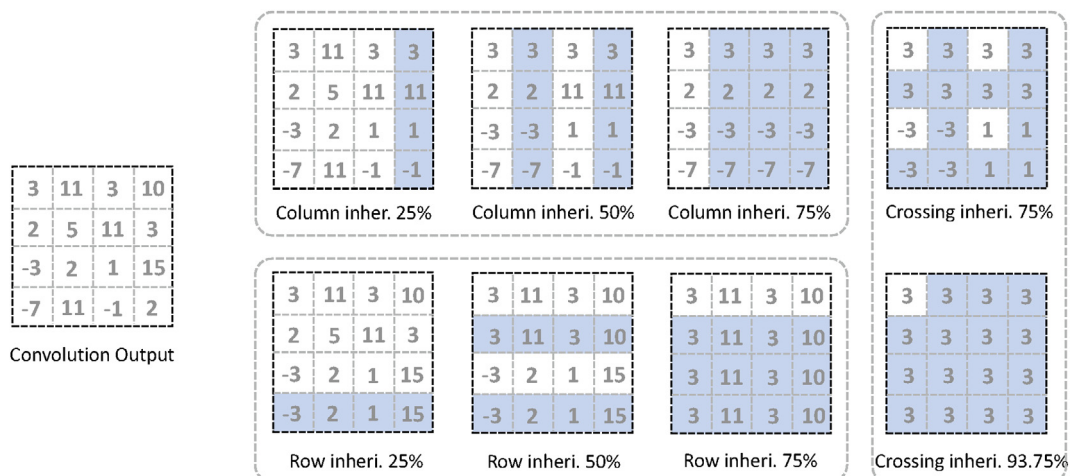


Fig. 2. Conv-Inheritance examples.

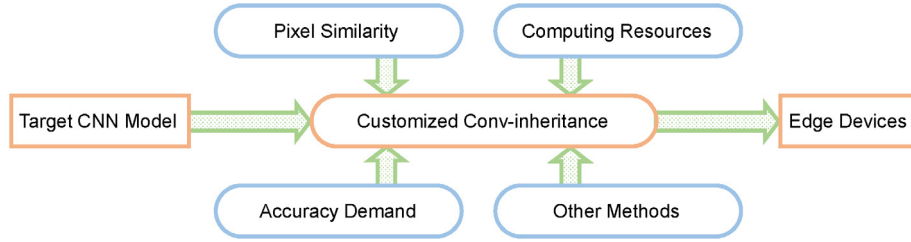


Fig. 3. Overview of Customized Conv-Inheritance.

limited, it is inevitable to arise accuracy loss when there are multiple compression methods. The more compression methods used, the lower the Conv-Inheritance ratio should be.

All in all, the first factor is positively associated with the inheritance ratios. And the last three factors are negatively associated with the inheritance ratios. The overview of deploying customized Conv-Inheritance is demonstrated in Fig. 3.

3.3. Basic Algorithm of Conv-Inheritance

Algorithm 1 shows the basic algorithm of Conv-Inheritance. In Algorithm 1, if the inheritance ratio (how many pixel values of the output features we utilize Conv-Inheritance to obtain) is more than 50%, we repeat the codes from line 2 to line 4 and change the values of k_0 and k_1 . Take a 75% inheritance ratio by only column inheritance as an example. First, we remove the conditional statement in line 2 and line 3 and set c equal to 4. Then we repeat line 4 three times by the value of k_1 from 1 to 3. Finally only in the case ' $j\%4 = 0$ ', we obtain the pixel value by the normal convolution operations and in the other three cases ' $j\%4 = 1$ ', ' $j\%4 = 2$ ', ' $j\%4 = 3$ ', we utilize the column inheritance.

In Algorithm 1, A_{ij} indicates the pixel value of the output image. k_0 and k_1 can be set as any integer value between 0 and r and between 0 and c .

Algorithm 1: Basic Algorithm of Conv-Inheritance

```

1: for each  $(i, j) \in \text{inputpicture}$  do
2:   if  $i\%r = k_0$  then  $A_{ij} = A_{(i-1)j}$  //about the row inheritance
3:   else
4:     if  $j\%c = k_1$  then  $A_{ij} = A_{i(j-1)}$  //about the column inheritance
5:   else  $A_{ij} = \text{conv}(\text{result from normal convolution operation})$ 
6:   /*Two conditional statements are about the crossing inheritance.*/
7:   end if
8: end if
9: end for

```

3.4. Matrix Rebuild Algorithm of Conv-Inheritance

Besides the basic algorithm of Conv-Inheritance, we propose the matrix rebuild algorithm of Conv-Inheritance for deep learning frameworks, because many deep learning frameworks transform the computation of convolution operation to the heavily-optimized matrix-by-matrix multiplication routine of basic linear algebra packages for high computational performance.

Fig. 4 illustrates the matrix version of the 3D convolution and Conv-Inheritance. The pixels of the input feature matrix in the red circle are the pixels calculated by Conv-Inheritance. In the matrix version of convolution, convolution operations are integrated into a matrix multiplication. In the input feature matrix, the same picture's pixel values are in the same column of the input feature matrix. The kernel values in the same kernel are in the same column of the kernels matrix. In the matrix rebuild algorithm of Conv-Inheritance, we reduce the input features matrix scale and obtain a small output features matrix. Then, we rebuild the output features matrix to restore the scale of the output features matrix. Algorithm 2 shows the details.

In Algorithm 2, r indicates the reciprocal of inheritance ratio. A_{ij} indicates the pixel value of the small output features matrix. k_0 can be set as any integer value between 0 and r . n indicates the row number of the input features matrix.

Algorithm 2: Matrix Rebuild Algorithm of Conv-Inheritance

```

1: for each  $i \in [1, n]$  do
2:   if  $i\%r = k_0$  then remove the row  $i$  from the input features matrix;
3:   end if
4: end for
5:  $\text{new input features matrix} \times \text{kernels matrix} = \text{small output features matrix}$ 
6: for each  $i \in [1, n]$  do
7:   if  $i\%r = k_0$  then add a row  $i$  to the small output features, for the pixels in row  $i$ ,  $A_{ij} = A_{(i-1)j}$ ;
8:   end if
9: end for

```

The two Conv-Inheritance Algorithms we discussed above are designed for different circumstances. Conv-Inheritance's basic algorithm can work for applications with the traditional convolution operations, such as small scale CNN edge devices. Conv-Inheritance's matrix rebuild algorithm is designed for the applications with the matrix version convolution operations, such as large scale CNN edge devices and deep learning frameworks including Caffe, Tensorflow, and Pytorch.

3.5. Adaptive Conv-Inheritance

As an extended version of our Conv-Inheritance, an adaptive Conv-Inheritance is presented by considering the similarity between different image pixels. In particular, two neighboring pixels are considered similar if the pixel difference between them is less than the threshold. Then, we utilize the Conv-Inheritance to gain the output value. Otherwise, we utilize the traditional convo-

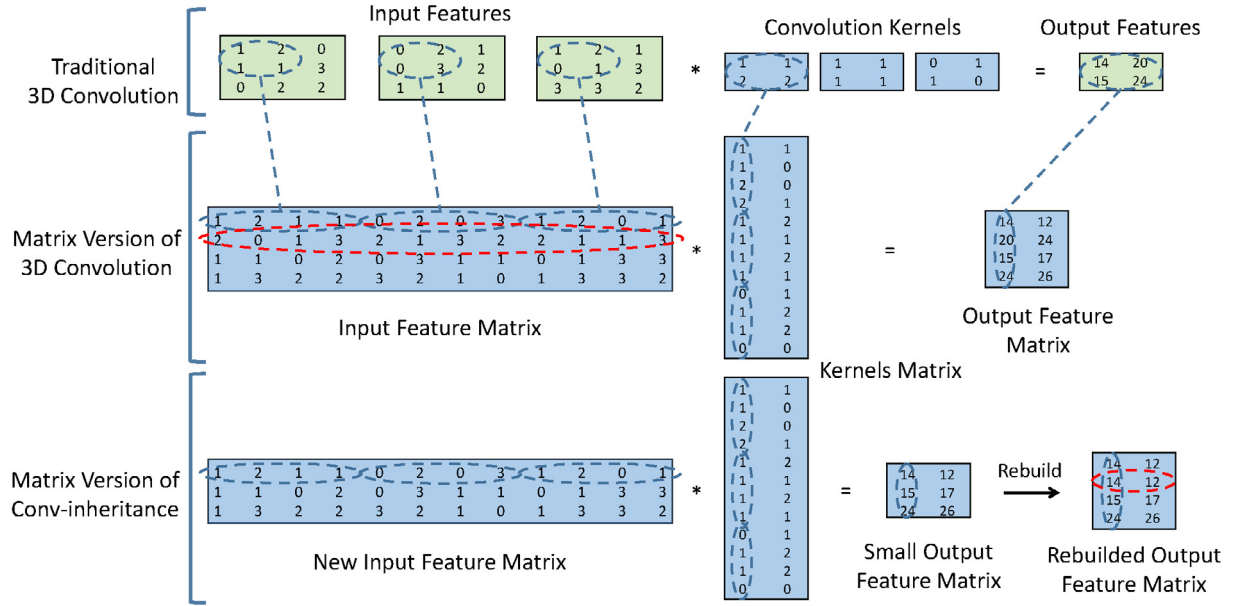


Fig. 4. Matrix Version of Convolution and Conv-Inheritance.

lution operation to gain the output value. Algorithm 3 shows the details of the adaptive Conv-Inheritance.

Algorithm 3:Algorithm of Adaptive Conv-Inheritance

```

1: for each  $P_{ij} \in \text{inputpicture}$ 
2:   if  $|P_{ij} - P_{i(j-1)}| < \text{threshold}$  then  $A_{ij} = A_{i(j-1)}$  (the Conv-Inheritance)
3:   else  $A_{ij} = \text{conv}$  (result from normal convolution operation)
4:   end if
5: end for

```

In the adaptive Conv-Inheritance method, it is difficult to determine the inheritance ratio accurately, as the numbers of similar neighboring pixels in the different inputs are varying. Therefore, we propose a probabilistic model to estimate the inheritance ratio according to the given threshold roughly.

We assume that each pixel value in the original input image obeys a uniform distribution from 0 to 255. Therefore, we can gain the expectation and variance of the input image pixels based on the formula for the uniform distribution, i.e., $\mu = \frac{a+b}{2}$ and $\sigma^2 = \frac{(a-b)^2}{12}$, where μ is the expectation, σ^2 is the variance, a is 0 and b is 255. When the input data is fed into the CNN models, they undergo batch normalization several times, which is mostly Z-score normalization as $\hat{x} = \frac{x-\mu}{\sigma}$, where x is the pixel value and $\hat{x}$ is the pixel value after the Z-score normalization. The data after Z-score normalization obeys the normal distribution, $f(x) = \frac{1}{\sqrt{2\pi}\sigma} * e^{-\frac{(x-\mu)^2}{2\sigma^2}}$ and $f(x)$ is the probability density function of the input pixel x . Given two neighboring pixels X_1 and X_2 , we can model the probability of their difference within the threshold by solving the probability in Eq. 1 below.

$$\text{Prob}(|X_1 - X_2| < \text{threshold}) = \Phi\left(\frac{\text{threshold}}{\sqrt{2}\sigma^2}\right), \quad (1)$$

Where X_1 and X_2 are independent and obey the same normal distribution, and $\Phi(\cdot)$ is the standard normal distribution function and its value can be referred to the standard normal distribution

probability table. Finally, we can estimate the inheritance ratio based on the Eq. 1. Moreover, our estimation method is based on the probability distribution hypothesis and pixel independence hypothesis. However, input images are various, and two neighboring pixels in pictures are usually similar to some extent in the nature images, which means X_1 and X_2 are actually not independent. So, the estimated inheritance ratio may be suitable for qualitative analysis. If the actual inheritance ratio is in need, we can count the inheritance times in the algorithm.

3.6. Hardware Efficiency

In practical applications, CNNs are often deployed on customized accelerators in edge devices whose platform includes GPU, FPGAs, and ASICs. The on-chip designs of customized accelerators are based on the features of CNN models. Generally, hardware designers design specific computing units for target CNN models to achieve high performance. For instance, the computing unit for MobileNet is different from the VGG's as a consequence of the difference between their convolution operations. Moreover, pipeline and parallelization are the core technologies in computing unit design for high throughput and speed. Thus, many computing

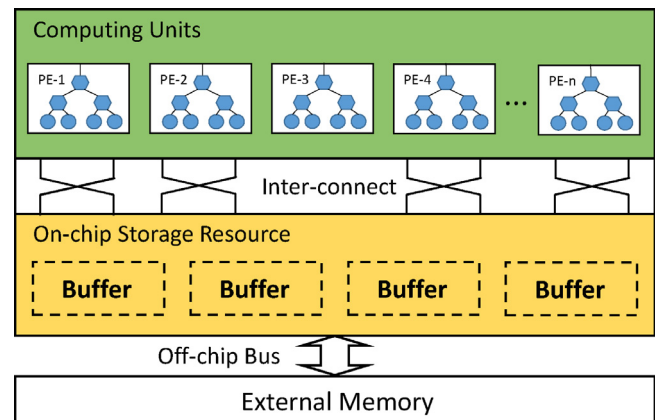


Fig. 5. Basic On-chip Architecture of Accelerators.

units are working simultaneously [29]. Fig. 5 illustrates the overview of the accelerator design.

A processing element (PE) is the basic computation unit for convolution. All data for processing are stored in external memory. Due to on-chip resource limitation, data is first cached in on-chip buffers before being fed to PEs. Multiple buffers are used to cover computation time with the data transfer time. The on-chip interconnect is dedicated to data communication between PEs and on-chip buffer banks. The hardware efficiency in this work means we utilize fewer hardware resources to achieve the CNN deployment.

Most CNN compression methods only decrease storage and computing requirements without considering the detailed design on the chip while compressing the CNN models. As a result, it requires lots of effort to redesign the hardware for deploying them. However, Conv-Inheritance can optimize the detailed design on chip as well as compressing CNN models. When utilizing Conv-Inheritance, we can replace some PEs with data reuse easily. Thus, it can reduce the efforts to deploy Conv-Inheritance on edge devices. The number of replaced PEs depends on the inheritance ratio. Taking 25% inheritance as an example, we can save one PE in every four PEs. Fig. 6 shows the details of the replacement.

4. Experiments

4.1. Conv-Inheritance performance in the approximate edge device environment

In our experiments, we demonstrate the speedup of Conv-Inheritance and the performance of different Conv-Inheritance types. In particular, the experiments are conducted on the MNIST dataset, and we utilize the LeNet as the neural network model for image recognition. Our experiment codes are written in the pure C language because the C language's running environment is very similar to the real operating environment of the edge devices, such as customized hardware accelerators. Moreover, it can also exclude the impact of frameworks such as Tensorflow on performance. The computer for the experiment is with the Windows 10 operating system and Intel(R) Core(TM) i7-9570H CPU. The RAM is 16 GB.

There are three types of Conv-Inheritance: column inheritance, row inheritance, and crossing inheritance in our experiment. The inheritance of column inheritance and row inheritance are 12.5%, 18.75%, 25%, 50% and 75%. The inheritance ratios of crossing inheritance are 23.4%, 34%, 43.75%, and 75%.

Fig. 7 illustrates the accuracy of three types of Conv-Inheritance, column inheritance, row inheritance, and crossing inheritance, in different inheritance ratios. In Fig. 7, we find that when the inheritance ratios are less than 50%, the accuracy of three types of Conv-Inheritance is almost equal, and the column inheritance has a little superiority. However, when it comes to high inheritance ratios (above 70%), the advantage of the crossing inheritance becomes

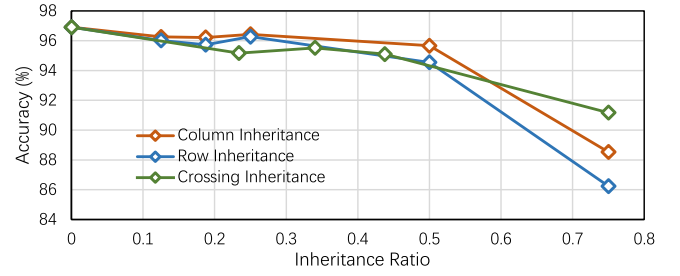


Fig. 7. Accuracy of Three Types of Conv-Inheritance.

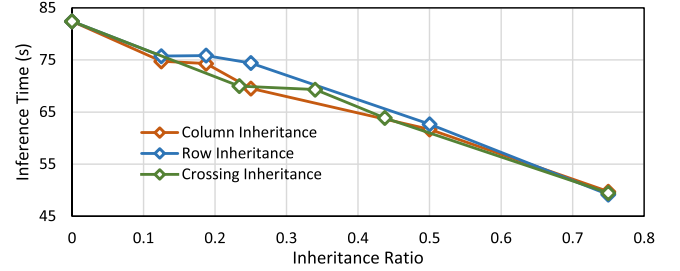


Fig. 8. Inference Time of Three Types of Conv-Inheritance.

evident. The crossing inheritance is about 5% better than the row inheritance and about 3% better than the column inheritance in the prediction accuracy.

Fig. 8 shows the inference time of three types of Conv-Inheritance, column inheritance, row inheritance, and crossing inheritance, in different inheritance ratios. According to the line chart, we figure out that the inference time is linearly varying with the inheritance ratio for all the three types of Conv-Inheritance. At the same inheritance ratio, the inference time gap of the three Conv-Inheritance methods is not very large in most cases.

To sum up, when the inheritance ratio is less than 50%, we suggest the column inheritance as an efficient Conv-Inheritance method for accelerating CNN inference. If the inheritance ratio is more than 70%, we believe the crossing inheritance is better than others.

4.2. Conv-Inheritance performance on popular datasets

In this experiment, we demonstrate the accuracy performance of Conv-Inheritance on popular datasets, which are the CIFAR10 dataset and the ImageNet dataset. For the CIFAR10 dataset, we utilize NIN [30] as the neural network model for recognition. For ImageNet dataset, we utilize the AlexNet [5], VGG-16 [6], DenseNet-121 [31] and ResNet-50 [8] as the neural network model for recognition. The framework we adopt for training and testing is Caffe. The Conv-Inheritance type is column inheritance. The

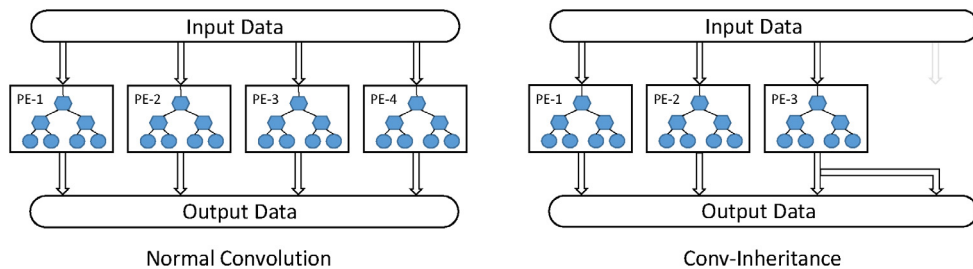


Fig. 6. Details of the replacement of PE.

Table 1

Accuracy(top-5) of Popular Datasets.

Inher. Ratio	ImageNet				CIFAR10
	AlexNet	VGG-16	DenseNet-121	ResNet-50	NIN
0%	78.52	90.5	92.39	90.63	87.48
5%	78.33	89.42	91.99	89.31	87.39
10%	78.28	89	91.41	87.64	87.27
20%	76.92	88	85.38	79.16	87.19
50%	75.44	80.5	64.31	59.78	84.52

Table 2

Accuracy(top-5) of ResNet-50 on ImageNet dataset.

Inher. RatioSparsity	0	50%	80%	90%
0%	90.63	90.25	89	86.39
5%	89.31	89.65	87.53	84.85
10%	87.64	87.34	86	82.81
20%	79.16	75.17	76	70.65

Table 3

Accuracy (top-5) on ImageNet dataset.

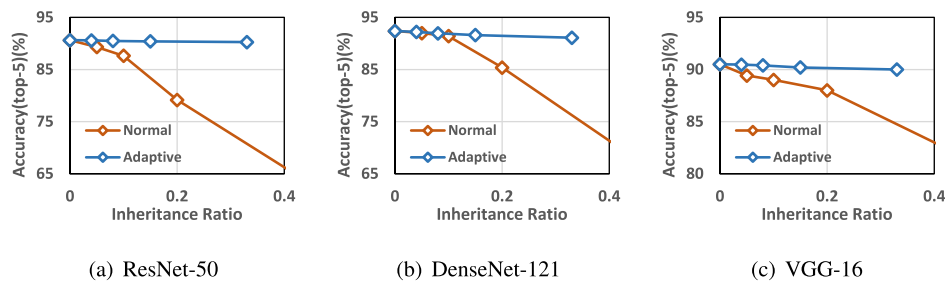
Threshold (ER)	ResNet-50	DenseNet-121	VGG-16
0(0%)	90.63	92.39	90.5
5(4%)	90.58	92.2	90.47
10(8%)	90.49	91.95	90.4
20(15%)	90.4	91.6	90.2
50(33%)	90.26	91.1	90

results of this experiment are in Table 1. Accuracy(top-5) on the ImageNet dataset means if the target label is one of our top 5 predictions, we think we predict this image correctly.

From Table 1, we can figure out that with the 5%, 10%, and 20% inheritance ratios, the accuracy loss is slight and acceptable. With the inheritance ratio of 50%, large models' performance is much better than lightweight ones because there is less redundancy in the lightweight models.

4.3. Conv-Inheritance performance utilized with other compression methods

In this experiment, we demonstrate the accuracy performance of Conv-Inheritance when being utilized with other compression methods, which are the Network Slimming [32] and pruning the weights of fully connected layers. Network Slimming reduces the model size by enforcing channel-level sparsity in the network. It prunes the channels whose weights are near zero by removing all their incoming and outgoing connections and corresponding weights. Fully connected layers pruning is similar to Network Slimming. The difference is that fully connected layer pruning prunes the weights of fully connected layers. In the experiment, we make the weights sparsity of these two compression methods at the same degree. We choose the ImageNet dataset, and we utilize the ResNet-50 as the neural network model. The framework we adopt for training and testing is Caffe. Our Conv-Inheritance type is column inheritance.

**Fig. 9.** Accuracy Comparison between Adaptive Conv-Inheritance and Normal Conv-Inheritance.**Table 4**

FF utilizations and operating latency of different inheritance ratios and hardware platforms

Inher. Ratio	Alveo U50		Ras. Pi 3	TX2
	FF Utilization	Latency(ms)	Latency(ms)	Latency(ms)
0%	2005	32.72	5.326	3.102
5%	1950	29.6	5.004	2.854
10%	1899	26.64	4.868	2.593
20%	1796	21.13	4.258	2.169
50%	1487	8.494	2.618	1.099

Table 2 shows the accuracy of CNN models with various weights sparsity on the ImageNet dataset in different inheritance ratios. From Table 2, we can observe that with the 5% and 10% inheritance ratios and within 50% sparsity, the accuracy loss is slight and acceptable.

4.4. Adaptive Conv-Inheritance performance

In this experiment, we demonstrate the accuracy of Adaptive Conv-Inheritance. We utilize the ResNet-50, DenseNet-121, and VGG-16 as the neural network models and run them on ImageNet. We adopt the Caffe framework for training and testing. Our Conv-Inheritance type is column inheritance.

Table 3 shows the accuracy of different CNN models on the ImageNet dataset using the different thresholds. The ER in the table refers to the estimated inheritance ratio for each threshold. From Table 3, we can observe that the accuracy loss is slight and acceptable.

Fig. 9 illustrates the accuracy of adaptive Conv-Inheritance and normal Conv-Inheritance in different inheritance ratios and CNN models. Note that the inheritance ratio of adaptive Conv-Inheritance is the estimated inheritance ratio based on Eq. 1. In Fig. 9, we find that adaptive Conv-Inheritance achieves better performance than normal Conv-Inheritance.

4.5. Performance of Conv-Inheritance on typical edge devices

In this section, we demonstrate the hardware efficiency of Conv-Inheritance on the real edge devices. We choose Alveo U50 (a typical FPGA accelerator), Raspberry Pi 3 (a typical CPU device), and NVIDIA Jetson TX2 (a typical GPU device) as the platforms and take the convolution operation of a 32×32 RGB picture and a $3 \times 3 \times 3$ kernel as the test program. Because the CPU and GPU devices adopt Von Neumann architecture (processor and memory structure), we take operating latency as the hardware usage evaluation metric. Besides, we take the FF (Flip Flop, a kind of computing resource of FPGAs) utilization and operating latency as the evaluation metrics for FPGA devices. The experiment data in different inheritance ratios are shown in Table 4. We can observe that Conv-Inheritance is both hardware-efficient and time-efficient.

5. Conclusion

In this paper, we investigated the problem of CNN compression and proposed a hardware-efficient CNN compression method by reducing the number of convolution operations. It can reduce both the inference time and computing resources when making the CNN deployment on edge devices. We further developed an extended version, named adaptive Conv-Inheritance, by considering the similarity between pixels. Moreover, Conv-Inheritance is a plug-and-play compression method for CNNs, and there are no conflicts between Conv-Inheritance and other compression methods. Experimental results demonstrate that Conv-Inheritance is a hardware-efficient and time-efficient compression method. It can compress the CNNs at the same time of optimizing the hardware design.

CRediT authorship contribution statement

Yang Yang: Conceptualization, Methodology, Software, Writing - original draft, Data curation. **Chao Wang:** Visualization, Investigation, Writing - review & editing, Supervision. **Lei Gong:** Investigation, Writing - review & editing. **Min Wu:** Visualization, Investigation, Writing - review & editing. **Xuehai Zhou:** Writing - review & editing, Supervision.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the National Key Research and Development Program of China (under Grant 2017YFA0700900, 2017YFA0700903), National Science Foundation of China (Grants 61976200, 61772482, 62172380, and 62102383), Jiangsu Provincial Natural Science Foundation (Grant BK20181193 and BK20210123), Youth Innovation Promotion Association CAS (No. 2017497), USTC Research Funds of the Double First-Class Initiative (Grant YD2150002005), and Fundamental Research Funds for the Central Universities (Grant WK2150110018).

References

- [1] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, You only look once: Unified, real-time object detection, in: *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [2] S. Ren, K. He, R. Girshick, J. Sun, Faster r-cnn: Towards real-time object detection with region proposal networks, in: *Advances in neural information processing systems*, 2015, pp. 91–99.
- [3] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [4] Y. Huang, Y. Cheng, D. Chen, H. Lee, J. Ngiam, Q.V. Le, Z. Chen, Gpipe: Efficient training of giant neural networks using pipeline parallelism, *arXiv preprint arXiv:1811.06965*.
- [5] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, in: *Advances in neural information processing systems*, 2012, pp. 1097–1105.
- [6] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, *arXiv preprint arXiv:1409.1556*.
- [7] C. Szegedy, Wei Liu, P. Yangqing Jia, S. Sermanet, D. Reed, D. Anguelov, V. Erhan, A. Rabinovich Vanhoucke, Going deeper with convolutions, in: *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1–9, <https://doi.org/10.1109/CVPR.2015.7298594>.
- [8] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [9] L. Gong, C. Wang, X. Li, H. Chen, X. Zhou, Maloc: A fully pipelined fpga accelerator for convolutional neural networks with all layers mapped on chip, *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37 (11) (2018) 2601–2612.
- [10] Y. Gong, L. Liu, M. Yang, L. Bourdev, Compressing deep convolutional networks using vector quantization (2014), *arXiv:1412.6115*.
- [11] Y. Chen, H. Fan, B. Xu, Z. Yan, Y. Kalantidis, M. Rohrbach, S. Yan, J. Feng, Drop an octave: Reducing spatial redundancy in convolutional neural networks with octave convolution, in: *The IEEE International Conference on Computer Vision (ICCV)*, 2019.
- [12] S. Han, H. Mao, W.J. Dally, Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding, *arXiv preprint arXiv:1510.00149*.
- [13] C. Wang, L. Gong, Q. Yu, X. Li, Y. Xie, X. Zhou, Dlau: A scalable deep learning accelerator unit on fpga, *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 36 (3) (2016) 513–517.
- [14] S. Miao, Z.J. Wang, R. Liao, A cnn regression approach for real-time 2d/3d registration, *IEEE Transactions on Medical Imaging* 35 (5) (2016) 1352–1363.
- [15] C. Wang, L. Gong, X. Ma, X. Li, X. Zhou, Wookong: A ubiquitous accelerator for recommendation algorithms with custom instruction sets on fpga, *IEEE Transactions on Computers* 69 (7) (2020) 1071–1082.
- [16] Y. He, J. Lin, Z. Liu, H. Wang, L.-J. Li, S. Han, Amc: Automl for model compression and acceleration on mobile devices, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 784–800.
- [17] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, D. Kalenichenko, Quantization and training of neural networks for efficient integer-arithmetic-only inference, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713.
- [18] F.N. Iandola, S. Han, M.W. Moskewicz, K. Ashraf, W.J. Dally, K. Keutzer, Squeezenet: Alexnet-level accuracy with 50x fewer parameters and < 0.5 mb model size, *arXiv preprint arXiv:1602.07360*.
- [19] A.G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, H. Adam, Mobilenets: Efficient convolutional neural networks for mobile vision applications, *arXiv preprint arXiv:1704.04861*.
- [20] X. Zhang, X. Zhou, M. Lin, J. Sun, Shufflenet: An extremely efficient convolutional neural network for mobile devices, in: *Proceedings of the IEEE*

Conference on Computer Vision and Pattern Recognition, 2018, pp. 6848–6856..

- [21] G. Huang, S. Liu, L. Van der Maaten, K.Q. Weinberger, Condensenet: An efficient densenet using learned group convolutions, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 2752–2761.
- [22] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen, Mobilenetv 2: Inverted residuals and linear bottlenecks, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 4510–4520.
- [23] B. Zoph, V. Vasudevan, J. Shlens, Q.V. Le, Learning transferable architectures for scalable image recognition, in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2018, pp. 8697–8710.
- [24] Y. Sun, B. Xue, M. Zhang, G.G. Yen, Automatically designing cnn architectures using genetic algorithm for image classification, arXiv preprint arXiv:1808.03818..
- [25] B. Wu, X. Dai, P. Zhang, Y. Wang, F. Sun, Y. Wu, Y. Tian, P. Vajda, Y. Jia, K. Keutzer, Fbnet: Hardware-aware efficient convnet design via differentiable neural architecture search, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2019, pp. 10734–10742..
- [26] H. Pham, M. Guan, B. Zoph, Q. Le, J. Dean, Efficient neural architecture search via parameters sharing, in: J. Dy, A. Krause (Eds.), Proceedings of the 35th International Conference on Machine Learning, Vol. 80 of Proceedings of Machine Learning Research, PMLR, Stockholmsmässan, Stockholm Sweden, 2018, pp. 4095–4104. url:<http://proceedings.mlr.press/v80/pham18a.html>.
- [27] C. Bucilua, R. Caruana, A. Niculescu-Mizil, Model compression (2006) 535.
- [28] G. Hinton, J. Dean, O. Vinyals, Distilling the knowledge in a neural network, NIPS, in, 2014.
- [29] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, J. Cong, Optimizing fpga-based accelerator design for deep convolutional neural networks, in: Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, ACM, 2015, pp. 161–170.
- [30] M. Lin, Q. Chen, S. Yan, Network in network, arXiv preprint arXiv:1312.4400..
- [31] G. Huang, Z. Liu, L. Van Der Maaten, K.Q. Weinberger, Densely connected convolutional networks, in: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 2261–2269, <https://doi.org/10.1109/CVPR.2017.243>.
- [32] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, C. Zhang, Learning efficient convolutional networks through network slimming, in: 2017 IEEE International Conference on Computer Vision (ICCV), 2017, pp. 2755–2763, <https://doi.org/10.1109/ICCV.2017.298>.
- [33] Chao Wang, Lei Gong, Xi Li, Xuehai Zhou, A Ubiquitous Machine Learning Accelerator With Automatic Parallelization on FPGA., IEEE Transactions on Parallel and Distributed Systems 31 (10) (2020) 2346–2359, <https://doi.org/10.1109/TPDS.2020.2990924>.
- [34] Qing Xu, Zhenghua Chen, Keyu Wu, Chao Wang, Min Wu, Xiaoli Li, KDnet-RUL: A Knowledge Distillation Framework to Compress Deep Neural Networks for Machine Remaining Useful Life Prediction, IEEE Transactions on Industrial Electronics 69 (2) (2021) 2022–2032, <https://doi.org/10.1109/TIE.2021.3057030>.



Yang Yang received the B.S. degree from the University of Science and Technology of China, Hefei, China, in 2015. He is currently pursuing the Ph.D. degree in computer science at the University of Science and Technology of China, Hefei, China. His current research interests include CNN model compression, knowledge distillation and deep learning accelerators.



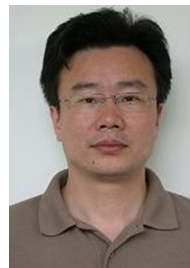
Chao Wang received the BS and PhD degrees from the University of Science and Technology of China, Hefei, China, in 2006 and 2011, respectively, both in computer science. He is currently an associate professor with the University of Science and Technology of China, Hefei, China. He was a visiting scholar with the Electrical and Computer Engineering Department, University of California at Santa Barbara, Santa Barbara, California, from 2015 to 2016. His research interests focus on multicore and reconfigurable computing. He serves as the associate editor of the ACM Transactions on Design Automations for Electronics Systems (ACM TODAES), IEEE/ACM Transactions on Computational Biology and Bioinformatics (TCBB), Microprocessors & Microsystems, and IET Computers & Design Techniques. He is also the publicity chair of HiPEAC 2015 and ISPA 2014. He is a senior member of the ACM, IEEE and CCF.



Lei Gong received Ph.D. degree in Computer Science from the University of Science and Technology of China, in 2019, and is currently a Postdoc with the School of Computer Science, University of Science and Technology of China. He participates in the research programs in Embedded System Lab, examining various aspects of embedded system with the focus on performance, availability, flexibility and energy efficiency. His research interests include Machine Learning Accelerators and FPGA-based systems. His paper has once been nominated as Best Paper Candidate in CODES+ISSS 2018.



Min Wu is currently a senior scientist at Institute for Infocomm Research, Agency for Science, Technology and Research (A*STAR), Singapore. He received his Ph.D. degree in Computer Science from Nanyang Technological University (NTU), Singapore, in 2011 and B.S. degree in Computer Science from University of Science and Technology of China (USTC) in 2006. He received the best paper awards in InCoB 2016 and DASFAA 2015. He also won the IJCAI competition on repeated buyers prediction in 2015. His current research interests include machine learning, data mining and bioinformatics. He is a senior member of the IEEE.



Xuehai Zhou received the B.S., M.S., and Ph.D. degrees from the University of Science and Technology of China, Hefei, China, in 1987, 1990, and 1997, respectively. He is a Professor with the School of Computer Science and the School of Software Engineering, University of Science and Technology of China. His current research interest includes various aspects of multicore and distributing systems. Mr. Zhou serves as a General Secretary of Steering Committee of Computer College Fundamental Lessons, and the Technical Committee of Open Systems, CCF.